

txt2tex Reference

Write math like you're at a whiteboard. Get L^AT_EX you can hand in.

<https://github.com/jmf-pobox/txt2tex>

Document Structure

=== Title ===	<code>\section *{Title}</code>
* Solution N **	Bold heading with spacing
(a) Text	Part label
TEXT: paragraph	Prose block (smart formula detection)
LATEX: <code>\cmd</code>	Raw L ^A T _E X passthrough
PAGEBREAK:	Page break
LINEBREAK:	Vertical gap (<code>\medskip</code>) — between blocks, no page break
TITLE: ...	Document title
AUTHOR: ...	Author name
DATE: ...	Date string
BIBLIOGRAPHY: <code>f.bib</code>	BibTeX file
[cite key]	<code>\citep {key}</code>

Identifier Decoration

<code>count'</code>	<i>count'</i> (after-state)
<code>in?</code>	<i>in?</i> (input)
<code>out!</code>	<i>out!</i> (output)
<code>x?'</code>	<i>x?'</i> (runs may combine)
<code>'sunk'</code>	'sunk' (string literal)

Decoration attaches to any identifier. Reserved words (`theta`, `defs`, `hide`, `project`, `Delta`, `Xi`) cannot be decorated.

Propositional Logic

<code>p land q</code>	$p \wedge q$
<code>p lor q</code>	$p \vee q$
<code>lnot p</code>	$\neg p$
<code>p => q</code>	$p \Rightarrow q$
<code>p <=> q</code>	$p \Leftrightarrow q$

Note: Use `land`, `lor`, `lnot`. English `and/or/not` are not supported.

Truth-table example:

You write:

TRUTH TABLE:			
p	q	p => q	
T	T	T	
T	F	F	
F	T	T	

F | F | T

You get:

p	q	$p \Rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Numbers & Arithmetic

<code>N</code>	$\mathbb{N}$ (natural numbers)
<code>Z</code>	$\mathbb{Z}$ (integers)
<code>N1</code>	$\mathbb{N}_1$ (positive naturals)
<code>n + m / n - m</code>	addition / subtraction
<code>n * m / n div m</code>	multiplication / division
<code>n mod m</code>	modulo
<code>n < m / n > m</code>	strict ordering
<code>n <= m / n >= m</code>	non-strict ordering
<code>n = m / n /= m</code>	equality / inequality

Sets & Types

<code>x elem S</code>	$x \in S$
<code>x notin S</code>	$x \notin S$
<code>A subset B</code>	$A \subseteq B$
<code>A union B</code>	$A \cup B$
<code>A inter B</code>	$A \cap B$
<code>A setminus B</code>	$A \setminus B$
<code>power S</code>	$\mathbb{P} S$
<code>F S</code>	$\mathbb{F} S$
<code>A cross B</code>	$A \times B$
<code>{x : S P}</code>	Set comprehension
<code>n..m</code>	$n \dots m$ (integer range)

Set comprehension example:

You write:

`{x : N | x > 0 land x < 5}`

You get:

$$\{x : \mathbb{N} \mid x > 0 \wedge x < 5\}$$

Predicate Logic

forall x : S | P $\forall x : S \bullet P$
exists x : S | P $\exists x : S \bullet P$
exists_1 x : S | P $\exists_1 x : S \bullet P$ (unique)
lambda x : S | E $\lambda x : S \bullet E$
mu x : S | P $\mu x : S \bullet P$ (definite desc.)
Bullet form: x : S | P means $x : S \bullet P$ (bullet is implied). The vertical bar is the quantifier separator.

Tuple-pattern:

You write:

```
forall (x, y) : N cross N |  
  x + y >= 0
```

You get:

$$\forall (x, y) : \mathbb{N} \times \mathbb{N} \bullet x + y \geq 0$$

Multi-decl: separate variable groups with ; inside a quantifier: forall x : A; y : B | P.

Schema-text scope: a bare [d : T | P] in a quantifier or function acts as an inline schema text.

Relations

A <-> B	$A \leftrightarrow B$ (relation type)
x -> y	$(x \mapsto y)$ (maplet)
dom R / ran R	$\text{dom } R / \text{ran } R$
A dres R	$A \triangleleft R$ (domain restrict)
A dsub R	$A \triangleleft R$ (domain subtract)
R rres B	$R \triangleright B$ (range restrict)
R rsub B	$R \triangleright B$ (range subtract)
R oplus S	$R \oplus S$ (relational override)
R o9 S	$R \S S$ (forward comp.)
R comp S	$R \S S$ (backward comp.)
R [A]	$R[A]$ (relational image)

Worked example — domain restrict:

You write:

```
{1, 2} dres {1 |-> a, 2 |-> b, 3 |-> c}
```

You get:

$$\{1, 2\} \triangleleft \{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\}$$

Functions

A pfun B	$A \leftrightarrow B$ (partial)
A fun B	$A \longrightarrow B$ (total)
A pinj B	$A \rightharpoonup B$ (partial injection)
A inj B	$A \rightharpoonup B$ (total injection)
A surj B	$A \twoheadrightarrow B$ (surjection)
A bij B	$A \twoheadrightarrow B$ (bijection)
A ffun B	$A \dashv\!\!\!\rightarrow B$ (finite partial)
f(x)	function application
lambda x : T f(x)	$\lambda x : T \bullet f(x)$

Worked example — lambda:

You write:

```
double == lambda n : N | n * 2
```

You get:

$$double == \lambda n : \mathbb{N} \bullet n * 2$$

Sequences

<> / <a, b>	$\langle \rangle / \langle a, b \rangle$
s ^ t	$s \frown t$ (concatenation)
seq S / seq1 S	$\text{seq } S / \text{seq}_1 S$
# s	$\#s$ (length)
head s / tail s	first / rest
last s / front s	last / all-but-last
rev s	reverse
s(i)	index (1-based)

Z Paragraphs

given A, B	Given types: $[A, B]$
T ::= a b	Free type definition
name == expr	Abbreviation

axdef / **schema** **Name** / **gendif** [**X**]
 declarations Variable declarations
 where Separator
 predicates Constraining predicates
 end Close the block

axdef introduces global constants; **gendif** [**X**] parameterises by a type; **schema** **Name** names a state or operation.

Schemas

A named schema introduces a signature (declarations) and an invariant (where clause).

You write:

```
schema Counter
  size : N
  nums : seq N
where
  size = # nums
end
```

You get:

<i>Counter</i>
<i>size</i> : $\mathbb{N}$ <i>nums</i> : seq $\mathbb{N}$
<i>size</i> = # <i>nums</i>

Where-clause patterns:

1. Simple comparison.

You write:

```
schema Bounded
  size : N
where
  size < 10
end
```

2. Set comprehension.

You write:

```
schema Inventory
  items : seq N
  stock : N
where
  stock = # {x : ran items | x > 0}
end
```

3. Quantifier.

You write:

```
schema NonNeg
  s : seq Z
where
  forall i : 1..#s | s(i) >= 0
end
```

Schemas in declarations.

A schema name used as a type pulls in that schema's signature:

You write:

```
schema Voyage
  s : Ship
  dest : Port
end
```

You get:

s inherits all fields of *Ship* as *s.field*

Generic schemas.

You write:

```
gendif [X]
  empty : F X
where
  empty = {}
end
```

You get:

$\langle X \rangle$ is the type parameter; instantiate with $G[T]$.

Schemas as predicates.

Inside a quantifier, a schema name acts as a predicate (both its signature and its where clause must be satisfied):

You write:

```
forall c : Counter |
  c.size >= 0
```

You get:

$\forall c : Counter \bullet c.size \geq 0$

Schema Operations

Inclusion operators.

SName	bare inclusion
Delta Counter	$\Delta Counter$ (before/after-state pair)
Xi Card	$\Xi Card$ (read-only state)
theta S	θS (binding of current state)
theta S'	$\theta S'$ (binding of after-state)

Horizontal definition.

Name defs RHS:	
Name defs Schema	$Name \hat{=} Schema$
N defs Delta C	$N \hat{=} \Delta C$
N defs [d : T P]	inline schema text
N[X] defs G[X]	generic form

Schema calculus operators (RHS of defs).

S[old/new]	rename component
S[a/b, c/d]	multiple renames
S ; T	$S \circ T$ (composition)
S » T	$S \gg T$ (piping)
S hide (x, y)	$S \setminus (x, y)$
S project T	$S \upharpoonright T$

Z Bindings

Z RM §3.7: binding brackets construct labelled tuples.

{ name == e }	$\langle name == e \rangle$
{ a == e, b == f }	multiple components
{ }	empty binding

Multi-typed comprehension with binding:

You write:

```
{ t : Track; a : Album |  
  t.albumId = a.albumId .  
  { | trackId == t.trackId | } }
```

You get:

$$\{ t : Track; a : Album \mid t.albumId = a.albumId \\ \bullet \langle trackId == t.trackId \rangle \}$$

Use ; to separate variable-type pairs inside { }. Binding components are **comma**-separated (Z RM §3.7).

Fuzz note: bindings emit outside any Z environment. Do not place them inside **zed/axdef/schema** blocks — fuzz rejects == inside $\langle \dots \rangle$ in block contents.

Scope rules. Identifiers declared in the left-hand signature of a comprehension are in scope in the predicate and the body expression.

Proofs

Truth-table format:

You write:

TRUTH TABLE:
p | q | p land q
T | T | T
T | F | F
F | T | F
F | F | F

You get:

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

Equivalence chain (EQUIV:):

You write:

EQUIV:
lnot (p land q)
lnot p lor lnot q [De Morgan]

You get:

$$\neg (p \wedge q) \\ \Leftrightarrow \neg p \vee \neg q \quad [\text{De Morgan}]$$

EQUIV: joins steps with $\Leftrightarrow$. Use for predicate equivalences.

Proofs (continued)

How proof trees work:

Proof trees are written **bottom-up**: the conclusion is the first line and premises are indented below it. Deeper indentation = deeper in the tree. Two premises at the same indent level are siblings that jointly support their parent. `::` marks sibling case-arms in a case analysis.

Proof tree syntax (PROOF:):

indent	Premises indented under conclusion
[rule]	Justification label
[rule [n]]	Discharge assumption n
[n] expr	Boxed assumption labelled n
::	Sibling premises

Rules: land intro, land elim 1/2, lor intro 1/2, lor elim, => intro/elim, <=> intro/elim, lnot intro/elim, forall intro/elim, exists intro/elim.

Modus ponens:

You write:

```
PROOF:
  q [=> elim]
  p => q
  p
```

You get:

$$\frac{p \Rightarrow q \quad p}{q} \Rightarrow \text{-elim}$$

$\wedge$ -intro:

You write:

```
PROOF:
  p land q [land intro]
  p
  q
```

You get:

$$\frac{p \quad q}{p \wedge q} \wedge \text{-intro}$$

$\Rightarrow$ -intro with discharge:

You write:

```
PROOF:
  p => p [=> intro [1]]
  [1] p
```

You get:

$$\frac{\lceil p \rceil^{[1]}}{p \Rightarrow p} \Rightarrow \text{-intro}^{[1]}$$

[1] p = boxed assumption. [=> intro [1]] discharges it.

$\vee$ -elim / case analysis:

You write:

```
PROOF:
  r [lor elim]
  p lor q
  :: p => r
    [1] p
      r [from p]
  :: q => r
    [2] q
      r [from q]
```

You get:

$$\frac{p \vee q \quad \frac{\lceil p \rceil^{[1]}}{r} \quad \frac{\lceil q \rceil^{[2]}}{r}}{r} \vee \text{-elim}$$

`::` = sibling premises (case arms).

Equality chain (EQUAL:):

You write:

```
EQUAL:
0 + 0
0      [0+0 = 0]
length(<>)
      [length(<>) = 0]
```

You get:

$$\begin{aligned} & 0 + 0 \\ & = 0 & [0+0 = 0] \\ & = \text{length}(\langle \rangle) & [\text{length}(\langle \rangle) = 0] \end{aligned}$$

EQUAL: = expression chains (=). **ARGUE:** = forward-reasoning argumentation blocks.

Relational Database Notation

Primary key annotation.

Mark a primary-key attribute with **pk** in a named schema body. The annotation sits outside the Z box so fuzz type-checks cleanly.

You write:

```
schema Book
  pk bookId : BookId
  isbn      : ISBN
  pages     : N
end
```

You get:

Book
bookId : *BookId*
isbn : *ISBN*
pages : *N*

$PK(Book) = \{bookId\}$

Composite: two **pk** lines $\Rightarrow PK(S) = \{a, b\}$. Comma-separated names on one **pk** line also work. **pk** is rejected in **axdef/gendef**/inline schema text.

Relational algebra.

sigma [p] (R)	$Restrict_p(R)$ — restrict
pi [A,B] (R)	$Project\{A, B\}(R)$ — project
rho [A as B] (R)	$Rename_{A \rightarrow B}(R)$ — rename
R bowtie S	$R \otimes S$ — natural join
R bowtie [p] S	$Join_p(R, S)$ — theta-join
R div S	$R \div S$ — division

sigma/pi/rho bind at atom level. **bowtie** and **div** are infix at cross-product precedence.

Fuzz note: algebra expressions emit outside any Z environment — fuzz silently skips them. Do not wrap algebra inside **zed/axdef/schema** blocks.

FK predicate form.

Foreign-key constraints are expressed as axdef predicates:

You write:

```
axdef
where
  (R, S) |-> {a |-> b} elem FK
end
```

GROUP & UNGROUP.

Date’s nested-relation operators.

R group ({A,B} as x)	$R \text{ GROUP}(\{A, B\} \text{ AS } x)$
R ungroup x	$R \text{ UNGROUP } x$

You write:

```
Members group ({name,email} as contact)
Members ungroup contact
```

You get:

$Members \text{ GROUP } (\{name, email\} \text{ AS } contact)$
 $Members \text{ UNGROUP } contact$

Fuzz note: GROUP/UNGROUP emit outside any Z environment. Do not place them inside **zed/axdef/schema** blocks.

Quick reference.

pk a : T	primary key in schema
pk a, b : T	composite pk
sigma [p] (R)	restrict
pi [A] (R)	project
rho [A as B] (R)	rename attr
R bowtie S	natural join ($\otimes$)
R div S	division
{ a == e }	binding bracket
{S; C P . E}	multi-typed set comp
R group ({A} as x)	group attrs
R ungroup x	ungroup attr